

PracticePro

Landing Page Audit Report

VEGA and Atrium: Claims vs. Implementation

Methodical cross-reference of every feature claim on both landing pages against the actual codebase implementation, identifying false claims, contradictions, overstated features, and verification gaps.

Date: 9 June 2026

Scope: Next.js marketing site + In-app Convex landing page

Codebase: /home/z/my-project/codebase_audit/pp/

Table of Contents

1. Executive Summary	4
1.1 Severity Definitions	4
2. VEGA Claims Audit	4
2.1 DraftPro Editor	4
2.2 ALOA AI Copilot	5
2.3 Auto-Format Rules	5
2.4 Party Grouping	6
2.5 Smart Placeholders	6
2.6 Matter Vault	6
2.7 Procedural Intelligence	6
2.8 Enterprise Jurisdiction Intake	7
2.9 Legacy Intelligence Activation	7
3. Atrium Claims Audit	8
3.1 Property Portfolio	8
3.2 Tenant Management	8
3.3 Rent Collection	8
3.4 Expense Tracking	9
3.5 Lease Management	9
3.6 Maintenance Tracking	10
3.7 Tenant Communication	10
4. Critical False Claims	10
4.1 Fabricated Institutional Endorsements	11
4.2 Non-Existent Enterprise Features	11
4.3 Non-Functional "Watch Demo" Button	11
5. Contradictions Between Landing Pages	12
5.1 Pricing Structure Contradiction	12
5.2 Uptime SLA Contradiction	12

5.3 Fabricated Usage Statistics	13
5.4 "No Hidden Fees" vs. Enterprise Setup Fee	13
6. Compliance and Trust Claims	13
6.1 NDPA 2023 Compliance	13
6.2 ISO 27001 / TLS 1.3 / AES-256	14
7. Consolidated Findings Summary	14
8. Recommendations	15
8.1 Immediate Actions (Before Next User Visit)	15
8.2 Short-Term Actions (Next Sprint)	16
8.3 Medium-Term Actions (Next Quarter)	16
9. Claims That Are Substantially True	17

1. Executive Summary

This report presents the results of a methodical, claim-by-claim audit of both the VEGA and Atrium landing pages for the PracticePro platform. Two separate landing page implementations were examined: the Next.js marketing site (`src/app/page.tsx`) and the in-app Convex landing page (`src/components/LandingPage.tsx`). Every feature claim, statistic, compliance badge, pricing line item, and trust endorsement was cross-referenced against the actual source code, Convex schema, backend functions, and component inventory.

The audit reveals **5 critical false claims**, **4 major contradictions** between the two landing pages, **12 overstated or unimplemented features**, and **6 unverified statistical claims** that appear fabricated. These findings range from trivial copy inconsistencies to serious legal and reputational risks, particularly around fabricated institutional endorsements and enterprise features that do not exist in the codebase.

1.1 Severity Definitions

Severity	Definition	Example
CRITICAL	Claim is provably false; legal/reputational risk	Institutional endorsements (NBA, Lagos Judiciary)
HIGH	Direct contradiction between two landing pages	Pricing structure completely different across pages
MEDIUM	Feature claimed but not fully implemented	Auto-Format Rules, true A4 pagination
WARNING	Feature exists but may not work as described	Procedural Intelligence "Live" badge
LOW	Claim is substantially true / verified	ALOA AI Copilot, Matter Management

2. VEGA Claims Audit

VEGA is positioned as the AI-powered legal drafting workspace for Nigerian law firms. The following subsections examine every specific claim made on both landing pages, assess whether the implementation supports the claim, and assign a severity rating where a gap exists.

2.1 DraftPro Editor

Claim (Next.js): "Rich-text editor with true A4 pagination, Nigerian legal fonts, and court-compliant formatting. What you see is what prints."

Actual Implementation: A TipTap-based WordProcessor component exists with custom extensions (LegalPlaceholder, LegalPartiesGroup, FontSize). The editor provides rich text editing and a header designer for letterheads. However, TipTap is a block-based editor that does not natively support "true A4 pagination" with real page breaks. There is no code in the WordProcessor that calculates page boundaries, inserts page-break markers at A4 dimensions, or renders a paginated preview. The "Nigerian legal fonts"

claim is also unsupported; the editor uses standard web fonts with no Nigerian-specific font registration. The claim "what you see is what prints" implies pixel-perfect screen-to-print fidelity, which is not achievable with a standard TipTap setup without a dedicated print rendering pipeline.

Verdict: MEDIUM. The rich-text editor exists and works, but "true A4 pagination" and "Nigerian legal fonts" are overstated. The pagination claim is technically misleading because the editor does not render page boundaries.

2.2 ALOA AI Copilot

Claim (Next.js): "AI-powered drafting assistant that understands Nigerian legal terminology, court rules, and document structures. Just ask."

Claim (In-App): "Your strategic AI partner. Analyze contracts, draft motions, and research strategy -- it understands your case files." Badge: "New"

Actual Implementation: ALOA AI assistant is fully implemented with a conversation system (`createAloaConversation`, `saveAloaMessage`), tool calling, action cards with completion tracking (`markAloaActionCompleted`), and a floating action button (FAB) for quick access. The AI is powered by Google Gemini through a secure server-side proxy (`convex/ai.ts`). It has access to firm data and can perform actions within the platform. The RAG (Retrieval-Augmented Generation) system using 768-dimension embeddings allows it to reference indexed firm documents. The claim of understanding "Nigerian legal terminology, court rules" is marketing language for the underlying Gemini model, but the integration is genuine and the tool-calling architecture is solid.

Verdict: LOW. Substantially true. The AI assistant is well-implemented. The "understands your case files" claim is supported by the RAG pipeline. The "New" badge is marketing, not a factual claim, so it does not constitute a misrepresentation.

2.3 Auto-Format Rules

Claim (Next.js): "One-click formatting for originating processes, affidavits, written addresses, and conveyances. Nigerian legal standards, automated."

Actual Implementation: There is no "Auto-Format Rules" feature anywhere in the codebase. The `WordProcessor` has formatting toolbar buttons (bold, italic, underline, etc.) and document templates with pre-defined structures, but there is no one-click mechanism that automatically reformats a document to comply with specific Nigerian court formatting rules. No function, component, or mutation handles automatic document reformatting for originating processes, affidavits, or conveyances. The closest feature is the document template system, which provides starting templates, but this is "template selection" not "auto-formatting."

Verdict: MEDIUM. This feature does not exist as described. Document templates provide a starting point, but there is no one-click auto-formatting engine. The claim should be revised to "Document templates for originating processes, affidavits, and conveyances" or the feature should be built.

2.4 Party Grouping

Claim (Next.js): "Structured claimant and respondent listings with bracketed numbering. Format parties exactly as the court requires."

Actual Implementation: The LegalPartiesGroup TipTap extension exists in the codebase, which provides structured grouping of parties within legal documents. Matters have a "parties" field that stores structured party data. The extension enables creating claimant/respondent groupings with proper formatting inside the editor.

Verdict: LOW. This claim is substantially true. The LegalPartiesGroup extension provides the described functionality.

2.5 Smart Placeholders

Claim (Next.js): "Fill-in-the-blank fields for variable content -- court names, party details, dates. Complete documents in minutes, not hours."

Actual Implementation: The LegalPlaceholder TipTap extension exists, providing fill-in-the-blank functionality within documents. Document templates with placeholders are available. The system supports variable content fields that can be populated with matter-specific data.

Verdict: LOW. Substantially true. The placeholder system is implemented and functional.

2.6 Matter Vault

Claim (Next.js): "Every document linked to its matter. Version history, access controls, and 7-year retention. Your evidence trail, protected."

Actual Implementation: Documents can be linked to matters via the matterId field. The FeatureGuard component provides role-based access control. However, there is no explicit "version history" implementation for documents; the Convex schema has no version or revision tracking fields on the documents table. The "7-year retention" claim refers to a data retention policy, but no automated retention enforcement or archiving mechanism exists in the codebase. The nightly purgeOldArchiveData cron cleans old archive records but does not enforce a 7-year retention window.

Verdict: MEDIUM. Document-matter linking and basic access controls exist, but "version history" and "7-year retention" are not implemented. These are specific, measurable claims that the codebase does not support.

2.7 Procedural Intelligence

Claim (In-App): "Access 40+ years of Supreme Court judgments and Nigerian court rules with AI-powered procedural extraction." Badge: "Live"

Actual Implementation: The infrastructure for legal content exists: a statutes table with 768-dimension vector embeddings, semantic search (searchStatutes), a legal modules catalog, and module licensing. The ALOA-X document indexing system can process legal documents. However, the claim of "40+ years of

Supreme Court judgments" pre-loaded into the system is unverified. The statutes table schema exists, but its population with actual Nigerian Supreme Court judgments spanning four decades is not evidenced in the codebase. The "Live" badge implies this feature is currently active and populated, which may be misleading if the content library is sparse or empty. There is no seed data, migration script, or content ingestion pipeline that populates this table with historical judgments.

Verdict: WARNING. The infrastructure exists, but the content claim ("40+ years of judgments") is unverified. If the statutes table is empty or sparsely populated, the "Live" badge is misleading. This needs verification against the production Convex deployment.

2.8 Enterprise Jurisdiction Intake

Claim (In-App): "Dynamic procedural intelligence engine validates court jurisdiction and rules. Seamlessly transition from matter creation into our drafting lab with all metadata pre-populated." Badge: "Updated"

Actual Implementation: The MatterIngestionWizard and SmartMatterModal exist for AI-assisted matter creation. The matters table has a jurisdictionalAnalysis field. The SmartPasteBox allows pasting case information for AI extraction. However, "validates court jurisdiction and rules" implies a rule engine that checks whether a particular court has jurisdiction over a matter type, which is not implemented. The jurisdictional analysis is AI-generated (via Gemini), not a validated procedural rules engine. The "seamlessly transition... with all metadata pre-populated" claim is partially supported: matter creation does carry over some fields, but the integration between matter creation and the WordProcessor drafting lab is not a seamless single-flow experience in the current UI.

Verdict: MEDIUM. The AI-assisted intake exists, but calling it a "procedural intelligence engine that validates court jurisdiction" overstates what is actually an AI-generated suggestion, not a validated rules engine.

2.9 Legacy Intelligence Activation

Claim (In-App): "Enterprise Concierge Service. We digitize your physical archives and ingest them into a private AI knowledge base." Steps: "1. Physical Digitization: We handle scanning, OCR, and indexing of your paper files." "2. R.A.G. Implementation: Your digitized history connects to ALOA."

Actual Implementation: The ALOA-X document indexing system (convex/indexer.ts) can process and index digital documents. The RAG pipeline (convex/embeddings.ts) stores and searches 768d embeddings. The useBrainAutoIndex hook automatically indexes firm documents. However, there is no scanning, OCR, or physical digitization capability in the application. The claim "We handle scanning, OCR, and indexing of your paper files" describes a physical concierge service that the software platform itself cannot perform. This is presented as a service offering rather than a software feature, but it is displayed on the self-service landing page without any qualification that it requires a separate engagement. No pricing, process, or timeline for this service is provided anywhere in the application.

Verdict: MEDIUM. The RAG technology exists, but the "physical digitization" claim describes a human-provided service that is not offered through the platform. This should be clearly marked as a

separate consulting engagement or removed until the service is operational.

3. Atrium Claims Audit

Atrium is positioned as the property management and revenue monitoring platform for Nigerian real estate portfolios. The following subsections examine each specific claim in detail.

3.1 Property Portfolio

Claim (Next.js): "Manage all your properties -- residential, commercial, and mixed-use -- from a single dashboard. Upload photos, track occupancy, and organize by location."

Actual Implementation: PropertyManagerView provides a property list with types (residential, commercial, mixed-use). Property images are supported through Convex file storage. Occupancy status is tracked per unit. The RevenueEngine provides a portfolio dashboard with KPIs. However, "organize by location" implies geographic filtering or map views, which are not implemented. Properties can be sorted and filtered by basic fields, but there is no location-based clustering, map integration, or geographic organization feature.

Verdict: LOW. Substantially true with a minor overstatement on the "organize by location" aspect, which implies map-based or geographic organization that does not exist.

3.2 Tenant Management

Claim (Next.js): "Complete tenant profiles with KYC, lease agreements, and communication history. Automated onboarding and offboarding workflows."

Actual Implementation: Tenants exist as contacts linked to properties and tenancies. The tenancies table tracks rent amounts, payment frequency, and arrears. AtriumInbox provides inbound message history. The AtriumPublicApplicationForm supports external tenant applications. However, "automated onboarding workflows" are limited to the application form; there is no multi-step onboarding pipeline, automated lease generation upon approval, or workflow state machine for bringing a tenant from application to active status. "Automated offboarding workflows" do not exist at all. There is no mechanism for lease termination, deposit return calculations, or move-out checklists. The contacts table has no offboarding state or process tracking.

Verdict: MEDIUM. Basic tenant tracking exists, but "automated onboarding and offboarding workflows" is significantly overstated. Onboarding is a single application form; offboarding is absent entirely.

3.3 Rent Collection

Claim (Next.js): "Collect rent in Naira with automated invoices, payment reminders, and receipt generation. Track payment status at a glance."

Claim (In-App): "Automated Rent Collection. Track payments, generate invoices, and reconcile accounts

automatically."

Actual Implementation: CollectRentModal exists for recording rent payments. The ledger_entries table provides an immutable financial ledger with rent, service charge, penalty, and deposit entry types. WhatsApp reminders are sent via Chakra Chat API with monthly quota enforcement. ReceiptDetailView exists. The daily sentry cron sends late notices for overdue charges. However, "automated invoices" implies that invoices are generated automatically when rent is due, which is not the case. Invoices exist in the billing system but are created manually. "Reconcile accounts automatically" is also overstated; there is no automated bank reconciliation or payment matching. The CollectRentModal is a manual entry form, not an automated payment collection system. Actual payment processing (receiving money from tenants) is not integrated; there is no Paystack, Flutterwave, or bank API integration.

Verdict: MEDIUM. Rent tracking and reminder infrastructure exists, but "collect rent" implies actual payment processing, and "automated invoices" implies auto-generation, neither of which is implemented. The system records payments; it does not collect them.

3.4 Expense Tracking

Claim (Next.js): "Log maintenance costs, service charges, and utility bills. Generate profit-and-loss statements per property or portfolio-wide."

Actual Implementation: Service charges (Diesel, Security, Cleaning, Water, Other) are tracked via the service_charges table with defaulter flagging. Maintenance history is stored on property records. The ledger system categorizes expenses. However, "generate profit-and-loss statements per property or portfolio-wide" is not implemented. The ReportingView exists with financial report categories, but there is no P&L; statement generator that calculates revenue minus expenses per property. The getCashFlowSummary query provides aggregate income and "revenue at risk" figures, but this is a cash flow summary, not a profit-and-loss statement with expense breakdowns, net income calculations, and period comparisons.

Verdict: MEDIUM. Expense logging exists, but P&L; statement generation is not implemented. The cash flow summary provides some financial visibility but is not equivalent to a P&L; statement.

3.5 Lease Management

Claim (Next.js): "Automated lease expiry alerts, renewal workflows, and rent review schedules. Never miss a critical date again."

Claim (In-App): "Automate renewals, rent increments, and track expiration dates effortlessly."

Actual Implementation: The tenancies table stores lease data including start/end dates and rent amounts. However, there are no automated lease expiry alerts. No cron job checks for upcoming lease expirations. No notification is generated when a lease is approaching its end date. There are no "renewal workflows"; renewal requires manually creating a new tenancy record. There are no "rent review schedules" or automated rent increment mechanisms. The calendar system has event reminders, but lease expiry dates are not automatically linked to calendar events or notifications.

Verdict: MEDIUM. Basic lease data storage exists, but every claim about automation (expiry alerts, renewal workflows, rent review schedules) is unimplemented. This is one of the most overstated feature areas.

3.6 Maintenance Tracking

Claim (In-App): "Track issues from report to resolution with assigned tasks for vendors and property managers."

Actual Implementation: The properties table has a maintenanceHistory field that stores maintenance records. The general tasks system supports assignment and status tracking. However, there is no dedicated maintenance tracking module with a "report to resolution" pipeline. Maintenance entries in property records are simple log entries, not tracked issues with status progression. There is no vendor management system, no task assignment to specific vendors, and no maintenance request submission interface for tenants. The AtriumInbox can receive maintenance messages from tenants, but these are not automatically converted into tracked maintenance tickets.

Verdict: MEDIUM. Basic maintenance logging exists, but the "report to resolution" pipeline with vendor assignment is not implemented. This is a general-purpose task system, not a property-specific maintenance tracking module.

3.7 Tenant Communication

Claim (Next.js): "Built-in messaging for maintenance requests, notices, and announcements. Keep everything documented and time-stamped."

Actual Implementation: AtriumInbox receives inbound WhatsApp/SMS/email messages from tenants. AI analysis of inbound messages provides intent, sentiment, and suggested replies. ComposeModal enables outbound messaging. WhatsApp integration via Chakra Chat API supports template messages, reminders, and late notices. Spam guard prevents duplicate messages. Messages are time-stamped and can be marked as read. However, there is no separate "maintenance requests" channel or category within messaging. Maintenance requests would arrive as general messages and are not automatically categorized or routed.

Verdict: LOW. Substantially true. The messaging infrastructure is well-implemented. The specific mention of "maintenance requests" as a message type is slightly misleading since messages are not categorized by type, but the functionality exists.

4. Critical False Claims

The following claims are provably false or represent the highest-risk misrepresentations. These require immediate correction to avoid legal exposure, loss of credibility, or consumer protection violations under Nigerian law.

4.1 Fabricated Institutional Endorsements

Claim (Next.js TrustBar): "Trusted by leading Nigerian legal institutions" with a marquee displaying: Nigerian Bar Association, Lagos State Judiciary, NDPA Compliant, NBA Digital Partner, FCT High Court, Rivers State Judiciary.

Analysis: There is no evidence anywhere in the codebase, documentation, or Convex deployment that PracticePro has partnerships, endorsements, or trust relationships with the Nigerian Bar Association, Lagos State Judiciary, FCT High Court, or Rivers State Judiciary. The "NBA Digital Partner" claim is particularly specific and serious; it implies a formal commercial or institutional partnership with the Nigerian Bar Association. If such a partnership does not exist, this constitutes a false endorsement claim that could trigger legal action under Nigerian consumer protection law and the Federal Competition and Consumer Protection Act (FCCPA) 2018. Displaying judicial bodies (Lagos State Judiciary, FCT High Court, Rivers State Judiciary) as "trusting" a commercial product is especially sensitive, as it implies government endorsement of a private service.

Verdict: CRITICAL. These endorsements appear fabricated. If real partnerships exist, they must be documented and the landing page should link to partnership announcements or MoUs. If not, this section must be removed immediately.

4.2 Non-Existent Enterprise Features

Claim (Next.js Enterprise Plan): "SSO and advanced security", "Custom integrations", "SLA guarantee", "On-premise option".

Analysis: None of these features exist in the codebase. SSO (Single Sign-On) requires SAML 2.0 or OIDC integration, neither of which is implemented; the current auth system uses email/password with PBKDF2 hashing. "Custom integrations" implies an API or webhook framework for third-party connections, which does not exist. "SLA guarantee" implies a published Service Level Agreement document with uptime commitments, penalty clauses, and enforcement mechanisms; no such document exists. "On-premise option" implies the ability to deploy PracticePro on a client's own infrastructure; the application is built on Convex (a managed cloud platform) and cannot be deployed on-premise without a complete architectural rewrite. These are not roadmap items hinted at; they are presented as available features of the Enterprise plan.

Verdict: CRITICAL. Four enterprise features are claimed that do not exist. Charging for features that are not implemented constitutes misrepresentation.

4.3 Non-Functional "Watch Demo" Button

Claim (Next.js Hero): "Watch Demo" button prominently displayed in the hero section.

Analysis: The button exists but does not link to any functional demo video or interactive demo experience. Clicking it performs no action. There is no demo video file, YouTube embed, or interactive demo walkthrough in the codebase. The in-app landing has a "Try Demo" button that does work (it activates a demo mode with pre-populated data), but the Next.js "Watch Demo" button is non-functional.

Verdict: HIGH. A prominent CTA button that does nothing erodes trust immediately. Either create a demo video and link it, or remove the button.

5. Contradictions Between Landing Pages

The Next.js marketing site and the in-app Convex landing page present different, and in some cases directly contradictory, information about the same product. These inconsistencies are not just aesthetic differences; they create genuine confusion for users comparing the two touchpoints.

5.1 Pricing Structure Contradiction

Attribute	Next.js Marketing Site	In-App Landing (VEGA)	In-App Landing (Atrium)
Tier count	3 (Free/Pro/Enterprise)	4 (Core/Growth/Pro/Enterprise)	4 (Starter/Growth/Pro/Enterprise)
Pro price	25,000/mo	80,000/mo	45,000/mo or 420,000/yr
Free tier name	Free	Core	Starter
Free tier limits	3 matters, 10 docs, 1 prop, 5 tenants	10 matters, unlimited data	15 units, 100 WhatsApp/mo
Includes both products?	Yes ("Every plan includes both VEGA and Atrium")	No (product-specific pricing)	No (product-specific pricing)

Verdict: HIGH. The pricing structures are fundamentally different across the three presentations. The Next.js site claims Pro at 25,000/mo while the in-app VEGA page shows 80,000/mo -- a 3.2x difference. The "every plan includes both products" claim on Next.js directly contradicts the separate product-specific pricing in the in-app landing. A user who signs up based on the marketing site pricing will encounter different prices in the actual application.

5.2 Uptime SLA Contradiction

Next.js Hero Stats: "98.7% Uptime SLA"

In-App VEGA Stats: "99.9% Platform Uptime"

In-App Atrium Stats: "99.9% Platform Uptime"

These two different uptime figures (98.7% vs 99.9%) represent fundamentally different SLA tiers. A 98.7% SLA allows approximately 4.7 days of downtime per year, while a 99.9% SLA allows only 8.76 hours. The discrepancy suggests these are aspirational numbers rather than measured metrics. Additionally, there is no SLA enforcement mechanism, monitoring dashboard, or uptime status page in the application.

Verdict: HIGH. Two different uptime figures on two customer-facing pages. Neither is measured or enforced. Choose one consistent figure and ensure it reflects actual performance.

5.3 Fabricated Usage Statistics

Stat	Next.js Hero	In-App VEGA	In-App Atrium	Evidence?
Users/Properties	2,400+ Legal Practitioners	10,000+ Documents Automated	5,000+ Properties Managed	None
Revenue/Volume	1.2B+ Rent Collected	40 yrs of Judgments Indexed	1B+ Rent Processed	None
Uptime	98.7%	99.9%	99.9%	None

Verdict: CRITICAL. No evidence exists for any of these statistics. The founder analytics dashboard (getFounderMetrics) could theoretically provide real numbers, but the landing pages use hardcoded static values. The "40 yrs of Judgments Indexed" stat has an asterisk in the in-app landing but no corresponding footnote. These should be replaced with real metrics from the Convex analytics system or removed.

5.4 "No Hidden Fees" vs. Enterprise Setup Fee

Next.js Pricing: "No hidden fees. No per-document charges. No surprises."

In-App Atrium Enterprise: "150,000 One-Time Setup Fee" (marked "Required")

A mandatory 150,000 setup fee is, by definition, a hidden fee if it is not disclosed on the marketing site where users first encounter pricing. The marketing site explicitly promises "no hidden fees" while the in-app page reveals a significant mandatory charge that is not disclosed upfront. This is not just a contradiction; it may violate Nigerian consumer protection regulations that require full price disclosure before purchase.

Verdict: HIGH. Direct contradiction between "no hidden fees" and a mandatory 150,000 setup fee. All fees must be disclosed on the marketing site.

6. Compliance and Trust Claims

6.1 NDPA 2023 Compliance

Claim (Next.js): "Built from the ground up for Nigeria's data protection regime. 7-year data retention, consent management, and audit trails come standard."

Claim (In-App): "NDPA Fully Compliant" badge

Actual Implementation: The NDPA compliance features are genuinely the strongest area of the platform. The codebase includes: consent recording (recordConsent mutation), email verification and OTP, brute-force lockout (5 attempts then 15-minute lock), MFA support, breach notification system (sendBreachNotification, triggerBreachNotification), cross-firm data isolation on all queries, server-side privacy projection (getFirmData strips heavy fields), and a data purge mechanism (purgeFirmData). The platform has a Privacy Policy page, Terms of Service, and Data Processing Agreement. However, the

"7-year data retention" claim is specific and not implemented as an automated policy. The `purgeOldData` cron cleans old records but does not enforce a 7-year minimum retention. The audit trail exists via activity logging and analytics events, but is not a formal audit log system as required by NDPA S.26(1)(c).

Verdict: WARNING. NDPA compliance infrastructure is strong, but the specific claim of "7-year data retention" and "audit trails" should be qualified. The platform has good data protection practices but calling it "fully compliant" without a formal Data Protection Impact Assessment (DPIA) and NDPA registration certificate is premature.

6.2 ISO 27001 / TLS 1.3 / AES-256

Claim (In-App TrustBadges): "ISO 27001 Aligned", "TLS 1.3 Encrypted", "AES-256 at Rest"

Actual Implementation: "ISO 27001 Aligned" means the platform follows some ISO 27001 principles but has not been certified. This is a defensible claim if the alignment is genuine, but it should not be confused with actual ISO 27001 certification. "TLS 1.3 Encrypted" is likely true as Convex uses HTTPS, though the specific TLS version depends on Convex's infrastructure configuration. "AES-256 at Rest" is a property of Convex's storage layer, which the platform inherits. These are technically accurate descriptions of the hosting platform's security properties rather than independent certifications of PracticePro itself.

Verdict: WARNING. The claims are technically defensible but could mislead users into thinking PracticePro itself is ISO 27001 certified. "Aligned" should remain prominent, and the distinction between "inherited" and "certified" should be clarified.

7. Consolidated Findings Summary

The following table summarizes every claim audited in this report, organized by severity. The total count reflects the breadth of the gap between marketing claims and implementation reality.

Claim	Severity	Source	Finding
DraftPro "True A4 Pagination"	MEDIUM	Next.js	Overstated: editor exists but no page-break rendering
DraftPro "Nigerian Legal Fonts"	MEDIUM	Next.js	Unimplemented: no Nigerian-specific font registration
Auto-Format Rules	MEDIUM	Next.js	Non-existent: no one-click auto-formatting for document types
Matter Vault "Version History"	MEDIUM	Next.js	Non-existent: no document version tracking in schema
Matter Vault "7-Year Retention"	MEDIUM	Next.js	Non-existent: no automated retention enforcement
Procedural Intelligence "40+ yrs"	WARNING	In-App	Unverified: infrastructure exists, content unconfirmed
Procedural Intelligence "Live"	WARNING	In-App	Potentially misleading: may be empty in production

Enterprise Jurisdiction Intake	MEDIUM	In-App	Overstated: AI suggestion, not validated rules engine
Legacy Intelligence "Physical Digitization"	MEDIUM	In-App	Non-existent: no scanning/OCR in app
Tenant "Automated Offboarding"	MEDIUM	Next.js	Non-existent: no offboarding workflow at all
Rent "Automated Invoices"	MEDIUM	Both	Overstated: invoices are manual, not auto-generated
Rent "Collect Rent" (payment processing)	MEDIUM	Next.js	Misleading: records payments, does not collect them
Expense "P&L; Statements"	MEDIUM	Next.js	Non-existent: cash flow summary is not a P&L; statement
Lease "Automated Expiry Alerts"	MEDIUM	Both	Non-existent: no cron, no notification for lease expiry
Lease "Renewal Workflows"	MEDIUM	Both	Non-existent: renewal requires manual new tenancy
Maintenance "Report to Resolution"	MEDIUM	In-App	Overstated: basic log entries, no pipeline or vendor mgmt
Institutional Endorsements (NBA, etc.)	CRITICAL	Next.js	Likely fabricated: no evidence of partnerships
Enterprise "SSO"	CRITICAL	Next.js	Non-existent: no SAML/OIDC implementation
Enterprise "On-Premise Option"	CRITICAL	Next.js	Impossible: Convex cloud-only architecture
Enterprise "Custom Integrations"	CRITICAL	Next.js	Non-existent: no API/webhook framework
Enterprise "SLA Guarantee"	CRITICAL	Next.js	Non-existent: no SLA document or enforcement
"Watch Demo" button	HIGH	Next.js	Non-functional: no demo video or walkthrough
Pricing structure contradiction	HIGH	Both	3 tiers vs 4 tiers, different prices across pages
"Every plan includes both" contradiction	HIGH	Both	Next.js says yes; in-app has separate pricing
Uptime 98.7% vs 99.9%	HIGH	Both	Different figures on different pages
"No hidden fees" vs 150K setup fee	HIGH	Both	Direct contradiction across pages
Usage statistics (2,400+, 1.2B+, etc.)	CRITICAL	Both	No evidence; likely fabricated
NDPA "7-Year Data Retention"	WARNING	Next.js	Not enforced: no automated retention policy
"ISO 27001 Aligned" clarity	WARNING	In-App	Defensible but could mislead; not certified

8. Recommendations

8.1 Immediate Actions (Before Next User Visit)

- **Remove fabricated institutional endorsements.** The TrustBar with NBA, Lagos State Judiciary, FCT High Court, and Rivers State Judiciary must be removed unless documented partnerships exist.

Replace with generic trust language ("Trusted by Nigerian legal practitioners") or real user testimonials.

- **Unify pricing across both landing pages.** Decide on one pricing structure and apply it consistently. If the in-app pricing is the authoritative version, update the Next.js marketing site to match. Remove the "every plan includes both VEGA and Atrium" claim or make it true.
- **Remove or qualify enterprise features.** Delete SSO, on-premise, custom integrations, and SLA guarantee from the Enterprise plan features list until they are built. Alternatively, add a "Coming Soon" or "Roadmap" qualifier.
- **Replace fabricated statistics with real metrics.** Pull actual numbers from the Convex analytics system (getFounderMetrics, getDashboardData) and display those. If the numbers are small, use relative growth metrics ("Growing 40% month-over-month") instead of inflated absolutes.
- **Fix the "Watch Demo" button.** Either link to an actual demo video or remove the button. The in-app "Try Demo" button works and should be the primary CTA on the marketing site as well.
- **Remove the "No hidden fees" claim** or disclose the 150,000 Enterprise setup fee on the marketing site pricing section.

8.2 Short-Term Actions (Next Sprint)

- **Downgrade "Auto-Format Rules" to "Document Templates".** The current template system is valuable; reframe the marketing copy to accurately describe it rather than claiming one-click auto-formatting.
- **Remove "True A4 Pagination" claims** until the WordProcessor implements actual page boundary rendering. Replace with "Professional document formatting" which is accurate.
- **Remove "7-Year Retention" and "Version History" from Matter Vault** until implemented. Replace with "Document-matter linking with secure storage" which is true.
- **Qualify lease management claims.** Change "Automated lease expiry alerts, renewal workflows, and rent review schedules" to "Track lease dates and tenancy terms" until automation is built.
- **Remove "P&L; Statements" from expense tracking.** Replace with "Track expenses and service charges across your portfolio" which accurately describes the current functionality.

8.3 Medium-Term Actions (Next Quarter)

- **Build lease expiry alerts.** Add a cron job that checks tenancy end dates and generates notifications for leases expiring within 30/60/90 days. This would validate the current claim.
- **Implement automated invoice generation.** Create a cron job that generates invoices when rent is due based on tenancy payment frequency. This would validate the "automated invoices" claim.
- **Build document version history.** Add a versions array to the documents schema and save a snapshot on each edit. This would validate the Matter Vault version history claim.
- **Populate the Procedural Intelligence content library.** Ingest Nigerian Supreme Court judgments into the statutes table and verify the "40+ years" claim before displaying a "Live" badge.

- **Implement true pagination in the WordProcessor.** Add page-break rendering at A4 boundaries with visual page separators. This would validate the "true A4 pagination" and "true pagination engine" claims.

9. Claims That Are Substantially True

To maintain balance, it is important to acknowledge the features that ARE implemented as claimed. PracticePro has genuine, substantive capabilities that form a strong foundation. The following claims passed the audit with minor or no issues:

Feature Claim	Source	Implementation Evidence
ALOA AI Copilot	Both	Full conversational AI with tool calling, RAG, and Gemini integration
Party Grouping	Next.js	LegalPartiesGroup TipTap extension implemented
Smart Placeholders	Next.js	LegalPlaceholder TipTap extension implemented
Matter Management	Both	Full CRUD with Kanban board, court, jurisdiction, billing model
Document Vault	Both	Upload, AI analysis, sharing, signature requests, vector search
Case Management (Kanban)	In-App	MatterBoardView with Kanban and list views
Property Portfolio Dashboard	Both	PropertyManagerView + RevenueEngine with KPIs
WhatsApp Integration	Both	Chakra Chat API, inbound webhook, quota enforcement, spam guard
Service Charge Tracking	In-App	Full cycle: create, flag defaulters, apply penalties, mark paid
Revenue Ledger	In-App	Immutable ledger entries with cash flow summary and trending
NDPA Compliance Core	Both	Consent recording, breach notification, data isolation, verification
Client Communication	In-App	Client portal, messaging, intake portal with AI recording
Legal Billing	In-App	Invoices, time tracking, expenses, billing view per matter
Tenant Communication	Next.js	AtriumInbox + WhatsApp + ComposeModal with AI analysis
Rent Demand Generation	In-App	generateRentDemand produces Markdown + LaTeX output
Vacancy Pipeline	In-App	Lead tracking: Inquiry to Vetted to Lease_Generated to Closed

The platform has a genuine and impressive foundation. The audit is not about tearing down what exists, but ensuring that what is promised matches what is delivered. The 16 features listed above represent real, working capabilities that the marketing can legitimately claim. The issue is the additional 29 claims that overstate, contradict, or fabricate functionality beyond what the codebase supports.